



EDUCAÇÃO A DISTÂNCIA

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Unidade 2 | Semana 3

Aritmética binária, números com sinal e complemento de 2



INSTITUTO FEDERAL
Sudeste de Minas Gerais

Educação a Distância
Pró-reitoria de Ensino

Curso Técnico em Desenvolvimento de Sistemas

Professor: Jeronimo Costa Penha

Apostila V2 | 2026



Apresentação

Na Semana 2, foi mostrado que valores podem ser escritos em diferentes bases. Nesta semana, passaremos a operar diretamente com esses padrões e a observar uma limitação fundamental do hardware: processadores e memórias trabalham com quantidades finitas de bits.

A aritmética binária não é apenas uma mudança de símbolos. Ela permite compreender como a ULA executa somas e subtrações, como o mesmo byte pode significar 241 ou -15, por que certos resultados “dão a volta” e como o processador registra condições como zero, *carry* e *overflow*.

IDEIA CENTRAL

Bits não possuem sinal por si mesmos. O tipo da operação e a regra de interpretação determinam o significado do resultado.

Objetivos de aprendizagem

- Realizar adições e subtrações binárias em largura fixa.
- Representar e interpretar inteiros com e sem sinal.
- Aplicar o complemento de dois para codificar números negativos e efetuar subtrações.
- Detectar *overflow* e distinguir as *flags* C e V.
- Relacionar operações binárias ao trabalho da ULA e dos registradores.

Roteiro de estudos

1. Assista ao primeiro vídeo e acompanhe as seções 1 a 4.
2. Refaça as operações sem consultar o resultado.
3. Assista ao segundo vídeo e estude as seções 5 a 11.
4. Resolva a exploração guiada e os exercícios.
5. Realize o questionário formativo mais de uma vez.

Distribuição sugerida do tempo

Atividade	Tempo
Duas videoaulas	~20 min
Leitura e exemplos	~2 h 10 min
Prática guiada	~55 min
Exercícios	~45 min
Questionário	35 min

1. O que significa calcular com bits?

Quando escrevemos 27 como 00011011_2 , escolhemos uma representação. Para realizar $27 + 45$, o processador não converte esses valores para símbolos decimais; um circuito somador combina os bits de cada coluna e transporta o excesso para a posição seguinte.

O resultado precisa caber em um registrador. Neste material usaremos principalmente 8 bits para tornar todas as posições didáticas. Processadores reais operam com larguras como 32 ou 64 bits, mas as mesmas regras continuam válidas.

ATENÇÃO

Sempre identifique três informações antes de calcular: largura em bits, interpretação com ou sem sinal e operação solicitada.

Largura fixa

Uma calculadora matemática pode exibir quantos dígitos forem necessários. Um registrador de 8 bits, porém, armazena somente oito bits. Se uma operação produzir um nono bit, ele poderá ser registrado como *carry-out*, mas não fará parte do valor armazenado naquele registrador.

Largura	Padrões	Maior valor sem sinal
4 bits	16	15
8 bits	256	255
16 bits	65.536	65.535
32 bits	4.294.967.296	4.294.967.295

2. Adição binária

A adição binária segue a lógica da adição decimal. A diferença é que cada coluna usa base 2: quando a soma chega a dois, escrevemos zero e transportamos um; quando chega a três, escrevemos um e transportamos um. Exemplos de soma em binário podem ser vistos na Figura 1.

Operação da coluna	Bit da soma	Carry-out
$0 + 0$	0	0
$0 + 1$ ou $1 + 0$	1	0
$1 + 1$	0	1
$1 + 1 + \text{carry-in}$	1	1

Adição binária: o transporte liga as colunas

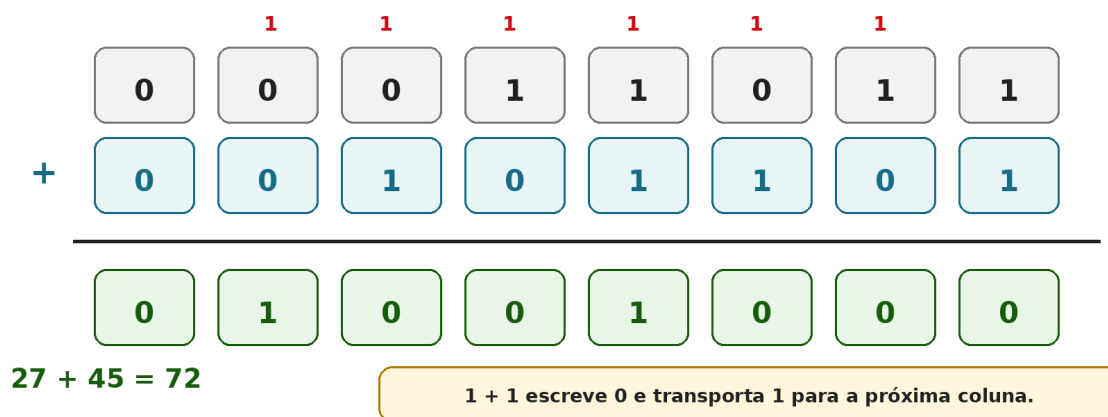


Figura 1: Exemplo de adição binária com transporte entre colunas.

Exemplo resolvido 1: 27 + 45

	1	1	1	1	1	1		← transportes
	0	0	0	1	1	0	1	1_2 (27)
+	0	0	1	0	1	1	0	1_2 (45)

	0	1	0	0	1	0	0	0_2 (72)

A conferência em decimal é $27 + 45 = 72$. Como 72 pertence ao intervalo de 0 a 255, não há estouro sem sinal.

Exemplo resolvido 2: resultado maior que 255

	1	1	1	1	1	1	1	← transportes
	1	0	1	0	1	1	0	1_2 (173)
+	0	1	0	1	1	0	1	1_2 (91)

	1	0	0	0	0	1	0	0_2 (264)

O valor 264 necessita de nove bits. Em um registrador de 8 bits permanecem 00001000_2 , isto é, 8. O bit à esquerda é o *carry-out* e a *flag C* é ativada.

ERRO COMUM

Não diga simplesmente que “a conta deu 8”. A soma matemática deu 264; 8 é o padrão que permaneceu após o truncamento para oito bits.

Lendo da direita para a esquerda após o empréstimo: $10_2 - 1_2$ produz 1; as duas colunas seguintes ficam com $1 - 1 = 0$; os bits de valores 8 e 16 permanecem em 1. Assim, o resultado é 00011001₂, que corresponde a 25.

O padrão 11110001_2 vale 241 se todos os bits tiverem pesos positivos. Em complemento de dois, o bit mais significativo tem peso -128, então o valor é $-128 + 64 + 32 + 16 + 1 = -15$. Exemplos podem ser vistos nas Figuras 2 e 3.

Essa distinção aparece em linguagens de programação, formatos de arquivo, instruções de máquina e protocolos. O hardware armazena o padrão; a instrução determina como tratá-lo.

Intervalos com 8 bits

SEM SINAL



COMPLEMENTO DE DOIS



Figura 3: Faixas representáveis com oito bits.

Fórmulas gerais

sem sinal: 0 até $2^n - 1$
complemento de dois: -2^{n-1} até $2^{n-1} - 1$

ASSIMETRIA

Em 8 bits com sinal existe -128, mas não existe +128. Há 128 valores negativos, o zero e 127 valores positivos.

5. Alternativas históricas para representar o sinal

Sinal e magnitude

O MSB indica o sinal e os demais bits representam o módulo. Em 8 bits, 00000101 seria +5 e 10000101 seria -5. O problema é a existência de +0 e -0, além da necessidade de tratar sinais separadamente nas operações.

Complemento de um

Um valor negativo é formado invertendo os bits do positivo. Assim, +5 é 00000101 e -5 é 11111010. Novamente aparecem dois zeros: 00000000 e 11111111.

Complemento de dois

O complemento de dois mantém apenas um zero e permite realizar $A - B$ como $A + (-B)$. Por isso, tornou-se a representação dominante de inteiros com sinal em processadores.

Representação	Como forma -5	Zeros	Observação
Sinal e magnitude	10000101	dois	sinal separado do módulo
Complemento de um	11111010	dois	inverte todos os bits
Complemento de dois	11111011	um	inverte e soma 1

6. Formando o complemento de dois

Como representar -37 em complemento de dois

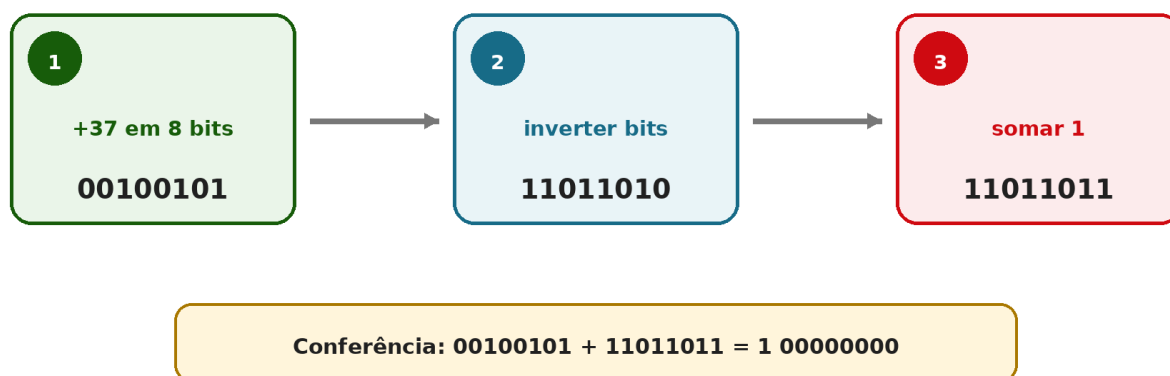


Figura 4: Procedimento para formar -37 com oito bits e verificação final: somando os dois, o resultado deve dar 0.

Método: inverter e somar 1

1. Escreva o módulo positivo na largura definida.
2. Inverta cada bit: 0 torna-se 1 e 1 torna-se 0.
3. Some uma unidade ao padrão invertido.
4. Mantenha somente a largura definida.

Exemplo resolvido: representar -37

+37	00100101

inversão	11011010
+1	+ 00000001

-37	11011011

Por que funciona?

Em n bits, o complemento de dois de x equivale a $2^n - x$. Para 8 bits, $256 - 37 = 219$. O número 219 sem sinal é 11011011_2 ; o mesmo padrão representa -37 quando lido com sinal.

CASO-LIMITE

Não tente formar +128 em 8 bits com sinal: esse valor não pertence ao intervalo. O padrão 10000000 representa -128.

7. Interpretando padrões negativos

Método 1: inverter e somar 1

Quando o MSB é 1, inverta o padrão, some uma unidade e atribua sinal negativo ao módulo obtido.

Padrão	11110001

inversão	00001110
+1	+ 00000001

módulo	00001111 = 15
valor	-15

Método 2: valor sem sinal menos 2ⁿ

O padrão 11110001₂ vale 241 sem sinal. Como a largura é 8, subtraímos 256: 241 - 256 = -15.

Padrão de 8 bits	Sem sinal	Complemento de dois
01111111	127	127
10000000	128	-128
10110110	182	-74
11011011	219	-37
11111111	255	-1

8. Adição e subtração com sinal

Exemplo resolvido: -37 + 22

	11011011	(-37)
+	00010110	(+22)

	11110001	(-15)

O resultado possui MSB 1. Invertendo 11110001, somando 1 e aplicando o sinal, obtemos -15.

Exemplo resolvido: 37 - 22

Primeiro formamos -22. O valor +22 é 00010110; invertendo e somando 1, obtemos 11101010.

$$\begin{array}{r} 00100101 \quad (+37) \\ + 11101010 \quad (-22) \\ \hline 1\ 00001111 \quad (+15) \end{array}$$

O *carry* externo é descartado e o resultado de oito bits é 00001111₂, equivalente a 15.

Exemplo resolvido: -20 - 15

$$\begin{array}{r} 11101100 \quad (-20) \\ + 11110001 \quad (-15) \\ \hline 1\ 11011101 \quad (-35) \end{array}$$

O resultado -35 pertence ao intervalo de -128 a 127. Portanto, não houve *overflow* assinado.

9. Overflow: quando o resultado não cabe

Overflow não significa que a ULA deixou de funcionar. Ela calculou os bits conforme a largura disponível; o problema é que o resultado matemático precisa de uma faixa maior.

Overflow assinado: o sinal denuncia o problema

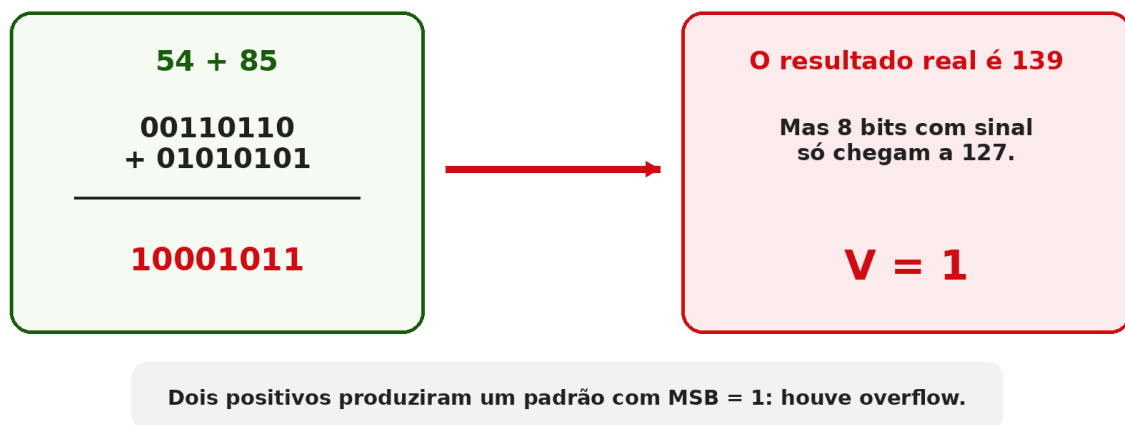


Figura 5: Exemplo de overflow em uma soma de dois valores positivos.

Regra prática para adição assinada

- positivo + positivo produz padrão negativo: *overflow*;
- negativo + negativo produz padrão positivo: *overflow*;
- operandos de sinais diferentes: não há *overflow* na adição.

Carry e overflow podem divergir

Operação de 8 bits	Leitura sem sinal	Leitura com sinal
11111111 + 00000001	255 + 1: <i>carry</i> , resultado 0	-1 + 1 = 0: sem <i>overflow</i>
01111111 + 00000001	127 + 1 = 128: sem <i>carry</i>	127 + 1: <i>overflow</i> , padrão -128
10000000 + 10000000	128 + 128: <i>carry</i> , resultado 0	-128 + -128: <i>overflow</i>

CONCLUSÃO

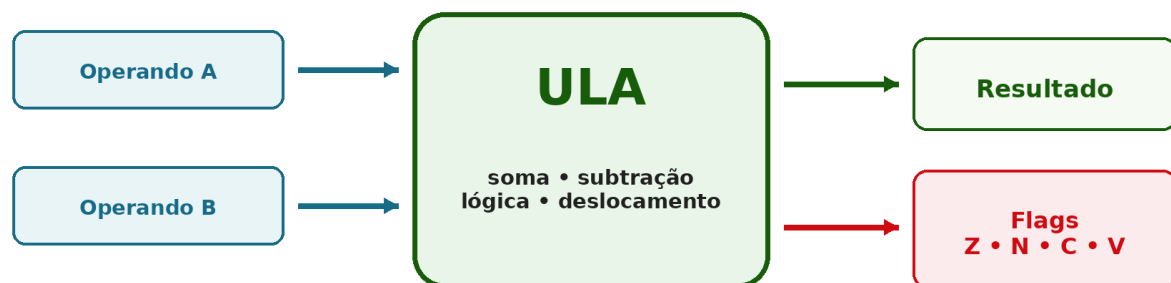
A *flag* C deve ser consultada para operações sem sinal; a *flag* V deve ser consultada para operações com sinal.

Overflow na subtração

Em $A - B$, ocorre *overflow assinado* quando A e B têm sinais diferentes e o sinal do resultado difere do sinal de A. Exemplo: $100 - (-50) = 150$, que não cabe em 8 bits com sinal (Figura 5).

10. A ULA e suas flags

ULA: resultado e flags saem da mesma operação



A instrução seguinte pode consultar as flags para decidir um desvio.

Figura 6: Entradas e saídas conceituais de uma ULA.

A unidade lógica e aritmética recebe operandos, executa a operação indicada pela unidade de controle e produz um resultado. Em paralelo, gera informações resumidas sobre esse resultado (Figura 6).

Flag	Nome	Quando é ativada
Z	Zero	todos os bits do resultado são 0
N	Negative	o MSB do resultado é 1
C	Carry	há transporte para fora da largura
V	Overflow	o resultado assinado não cabe na faixa

Por que *flags* importam?

Uma instrução de comparação pode executar internamente uma subtração e, em seguida, consultar *flags* para decidir se outro trecho do programa deve ser executado. Assim, operações de alto nível como “se A for menor que B” dependem de resultados e condições produzidos pela ULA.

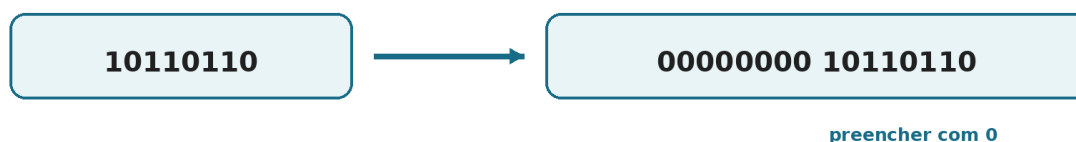
CUIDADO

A *flag* N informa que o MSB do padrão é 1; se ocorreu *overflow*, esse sinal pode não representar o sinal matemático correto.

11. Extensão de zero e extensão de sinal

Ampliar sem mudar o valor

SEM SINAL



COM SINAL

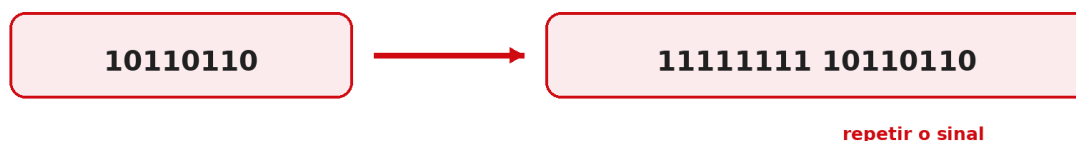


Figura 7: Ampliação de um padrão de 8 para 16 bits.

Para ampliar um inteiro sem sinal, novas posições à esquerda recebem zero. Para ampliar um inteiro com sinal, repetimos o MSB. Esse procedimento é chamado extensão de sinal (Figura 7).

Exemplo

O padrão 10110110 representa 182 sem sinal e -74 com sinal. Ao ampliar para 16 bits:

```
sem sinal:  00000000 10110110 = 182
com sinal:  11111111 10110110 = -74
```

Preencher um valor negativo com zeros mudaria seu significado. Por isso, a instrução e o tipo de dado precisam determinar a extensão correta.

12. Relação com programas

Linguagens e plataformas lidam de maneiras diferentes com limites numéricos. Em *hardware*, registradores têm largura fixa. Em linguagens como C, tipos inteiros possuem larguras e interpretações específicas. Em Python, inteiros comuns podem crescer para utilizar mais memória, ocultando muitos *overflows* que ocorreriam diretamente em registradores fixos.

Mesmo quando a linguagem protege o programador, formatos de arquivos, imagens, redes, bancos de dados e dispositivos continuam usando campos de tamanho limitado. Portanto, compreender faixa, sinal e *overflow* ajuda a evitar erros de conversão, comparação e armazenamento.

Situação	Pergunta que o desenvolvedor deve fazer
Campo de 8 bits	O valor é com ou sem sinal?
Conversão de tipo	A nova largura comporta todos os valores?
Leitura de arquivo	Qual codificação e ordem de bytes foram usadas?
Comunicação com dispositivo	Qual é a largura do registrador ou protocolo?
Resultado inesperado	Houve truncamento, <i>carry</i> ou <i>overflow</i> ?

13. Estratégia para resolver exercícios

1. Identifique a largura: 4, 8, 16 ou outra quantidade de bits.
2. Identifique a interpretação: sem sinal ou complemento de dois.
3. Converta os operandos e confira se pertencem ao intervalo.
4. Execute a operação mantendo todas as colunas alinhadas.
5. Separe o padrão armazenado do *carry-out*.
6. Interprete o resultado na regra solicitada.
7. Verifique *carry* ou *overflow* conforme o tipo.
8. Faça uma conferência em decimal.

REGRAS DE OURO	Nunca conclua que o MSB 1 significa erro. Ele pode representar um valor negativo válido. O erro só existe quando o resultado matemático não cabe no intervalo.
-----------------------	--

14. Exploração guiada

Use papel, uma planilha ou a calculadora do sistema operacional em modo programador. Para cada item, registre operandos, padrão armazenado, interpretação decimal e *flags* relevantes.

Operação em 8 bits	Resultado armazenado	C	V
00110101 + 00011110	_____	—	—
11111111 + 00000001	_____	—	—
01111111 + 00000001	_____	—	—
11010011 + 00010010	_____	—	—
00101101 - 00011100	_____	—	—

Perguntas de análise

1. Em quais operações *carry* e *overflow* tiveram valores diferentes?
2. Qual resultado seria correto sem sinal e incorreto com sinal?
3. Qual resultado seria correto com sinal e incorreto sem sinal?
4. Como cada operação se relaciona com o intervalo de 8 bits?

15. Exercícios de consolidação

Parte A: aritmética sem sinal

1. Some 00110101_2 e 00011110_2 .
2. Subtraia 00100111_2 de 01100100_2 .
3. Some 11110000_2 e 00100000_2 . Indique o *carry-out*.
4. Desloque 01011010_2 uma posição à esquerda.
5. Desloque 10010111_2 uma posição à direita como valor sem sinal.

Parte B: complemento de dois

1. Represente -18, -45 e -100 em 8 bits.
2. Interprete 11101110_2 , 11010011_2 e 10010110_2 com sinal.
3. Calcule $-45 + 18$ em 8 bits.
4. Calcule $37 - 52$ utilizando complemento de dois.
5. Estenda 10110110_2 de 8 para 16 bits preservando a interpretação com sinal.

Parte C: overflow e flags

1. Determine se $90 + 60$ produz *overflow* com sinal.
2. Determine C e V em $11111111_2 + 00000001_2$.
3. Determine C e V em $01111111_2 + 00000001_2$.
4. Explique por que *carry* e *overflow* não são sinônimos.
5. Explique como uma comparação pode utilizar *flags* produzidas pela ULA.

16. Respostas comentadas

Parte A

Item	Resposta
1	$01010011_2 = 83$
2	$00111101_2 = 61$
3	00010000_2 armazenado; <i>carry-out</i> = 1
4	$10110100_2 = 180$; não houve perda
5	$01001011_2 = 75$

Parte B

Item	Resposta
1	$-18 = 11101110$; $-45 = 11010011$; $-100 = 10011100$
2	$11101110 = -18$; $11010011 = -45$; $10010110 = -106$
3	$-45 + 18 = -27 = 11100101$
4	$37 - 52 = -15 = 11110001$
5	11111111 10110110

Parte C

Item	Resposta comentada
1	$90 + 60 = 150$; <i>overflow</i> , pois excede 127 e dois positivos geram MSB 1.
2	$C = 1$ e $V = 0$: sem sinal há estouro; com sinal, $-1 + 1 = 0$.
3	$C = 0$ e $V = 1$: sem sinal, $127 + 1 = 128$; com sinal, excede 127.
4	C observa transporte sem sinal; V observa se a soma assinada cabe no intervalo.
5	A ULA pode subtrair operandos e usar Z, N, C e V para orientar uma instrução de desvio.

17. Resumo

- A soma binária transporta uma unidade quando a coluna alcança a base 2.
- A largura fixa limita o conjunto de padrões armazenáveis.
- Sem sinal, n bits representam 0 a $2^n - 1$.
- Em complemento de dois, n bits representam -2^{n-1} a $2^{n-1} - 1$.
- Para formar um negativo, invertem-se os bits e soma-se 1.
- A subtração pode ser executada como soma do complemento de dois.
- *Carry* e *overflow* correspondem a interpretações diferentes.
- *Flags* conectam resultados da ULA às decisões das instruções.

Checklist de domínio

- Consigo somar e subtrair padrões binários.
- Consigo representar um número negativo em complemento de dois.
- Consigo interpretar um padrão cujo MSB é 1.
- Consigo determinar o intervalo para uma largura qualquer.
- Consigo diferenciar C de V e identificar overflow.
- Consigo explicar a relação entre ULA, registradores e flags.

Glossário

Termo	Definição
<i>Carry-in</i>	transporte recebido da coluna menos significativa
<i>Carry-out</i>	transporte produzido para fora de uma coluna ou da largura
Complemento de dois	representação de inteiros com sinal que usa inversão e soma de 1
Largura	quantidade fixa de bits de um registrador ou campo
<i>Overflow</i>	resultado matemático fora do intervalo representável
Truncamento	descarte dos bits que não cabem na largura
<i>Flag</i>	indicador produzido pela ULA sobre o resultado
Extensão de sinal	repetição do MSB ao ampliar um valor assinado

Referências

HENNESSY, John L.; PATTERSON, David A. Arquitetura de Computadores: uma abordagem quantitativa. Rio de Janeiro: Grupo GEN, 2019.

PATTERSON, David A.; HENNESSY, John L. Organização e Projeto de Computadores: a interface hardware/software. Rio de Janeiro: Elsevier, 2017.

WEBER, Raul Fernando. Fundamentos de Arquitetura de Computadores. Rio de Janeiro: Grupo A, 2012.

Material elaborado para a disciplina Arquitetura e Organização de Computadores: IF Sudeste MG, 2026.